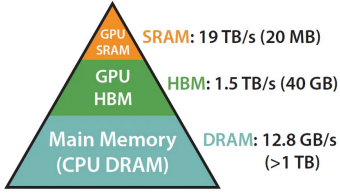
Deep Learning for NLP

M Pre-Training
How to reduce compute and memory



Learning goals

- Learn about distributed training with data/tensor parallelism
- Learn about FlashAttention

WHY DISTRIBUTE LLM TRAINING? I

Training large models on one GPU has two problems:

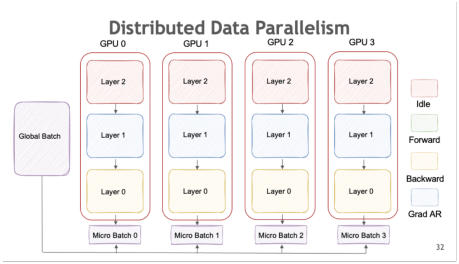
- **Speed:** training may take too long.
- **Memory:** the model and training state may not fit.

Using several GPUs can address both, but the computation must be distributed appropriately.
Two basic strategies are:

- **Data parallelism:** replicate the model and split the data.
- **Tensor parallelism:** split the model itself across GPUs.

Different strategies solve different bottlenecks.

DATA PARALLELISM: SAME MODEL, DIFFERENT DATA I



Source: Nvidia

Each GPU

- stores a **full copy of the model**,
- processes a different part of the batch,
- computes gradients locally.

The gradients are then averaged across GPUs (**All Reduce**).
All GPUs apply the same update, so their model copies remain identical.

WHAT DOES DATA PARALLELISM BUY US? I

Different GPUs process different training examples **in parallel**.
Therefore, data parallelism can increase training throughput:

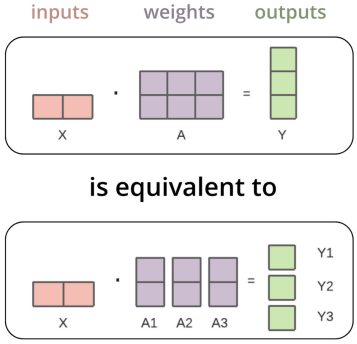
more GPUs $\implies$ more examples processed per second.

But every GPU still stores the **entire model**.

- Good for reducing training time.
- Does **not** allow a model that is too large for one GPU to fit.
- Gradient synchronization adds communication overhead.

To split the model itself, we need tensor parallelism.

TENSOR PARALLELISM: SPLIT THE MODEL ACROSS GPUS I



Source: Nvidia

Instead of replicating the full model, split large tensors across GPUs.
For a matrix multiplication

$$Y = XW,$$

the weight matrix W can be partitioned across devices.
Each GPU

- stores only part of W,
- computes part of Y,
- exchanges intermediate results when needed.

DATA VS. TENSOR PARALLELISM I

	Data parallelism	**Tensor parallelism**
Model	replicated	split across GPUs
Data	split	shared/processed jointly
Main benefit	higher throughput	larger models fit
Communication	between updates	within layers

- Data parallelism is useful when the model fits on one GPU, but training is too slow.
- Tensor parallelism is useful when individual model operations are too large for one GPU.

Large-scale training commonly combines both strategies.

WHY IS ATTENTION EXPENSIVE IN MEMORY? I

For a sequence of n tokens, attention computes

$$S = QK^\top, \qquad P = \mathrm{softmax}(S), \qquad O = PV.$$

Both intermediate matrices contain one value for every pair of tokens:

$$S, P \in \mathbb{R}^{n \times n}.$$

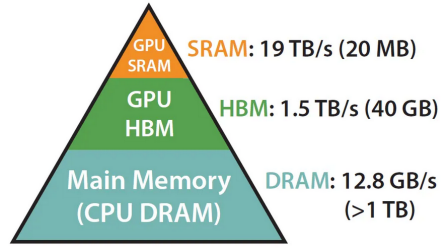
Hence, storing them requires

$$\boxed{O(n^2) \text{ memory.}}$$

Doubling the sequence length makes these matrices about **4 times larger**.

Can we compute exact attention without storing these large intermediate matrices?

THE GPU MEMORY HIERARCHY I



▸ Source: Dao et al., 2022

GPU memory has a hierarchy:

- **Static Random-Access Memory (SRAM):** very fast, but small
- **High-Bandwidth Memory (HBM):** much larger, but slower

GPU compute reads from and writes to SRAM.
Moving data and results between HBM and SRAM can become the bottleneck.

Reducing this memory traffic can make computation faster.

WHY STANDARD ATTENTION MOVES SO MUCH DATA I

A standard GPU implementation typically performs:

$$Q, K \longrightarrow S = QK^\top \longrightarrow P = \mathrm{softmax}(S) \longrightarrow O = PV.$$

Between these operations,

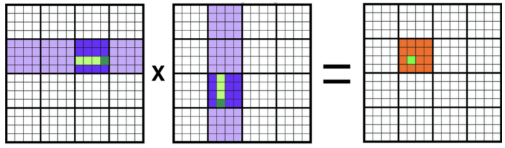
1. Q, K are read from HBM; S is computed and written to HBM,
2. S is read back; P is computed and written to HBM,
3. P, V are read back to compute O.

Thus, the large $n \times n$ matrices S and P are repeatedly transferred between HBM and SRAM.

FlashAttention avoids materializing S and P in HBM.

▸ Dao et al., 2022

FLASHATTENTION: COMPUTE ATTENTION BLOCK BY BLOCK I



Take one query block Q_i and process all key/value blocks in turn:

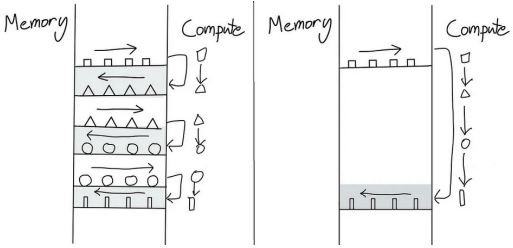
$$(K_1, V_1),\ (K_2, V_2),\ \ldots$$

For each block j:

1. load Q_i, K_j, V_j from HBM into SRAM,
2. compute $S_{ij} = Q_i K_j^\top$,
3. update the running softmax normalization,
4. update the partial output O_i, discard S_{ij} and its softmax values.

After the final block, O_i is normalized and written to HBM. All blocks are processed, but not stored at the same time.

FLASHATTENTION: AN ILLUSTRATION I



Source: horace.io

During the **forward pass**, each score block S_{ij} and its softmax probabilities are used and then discarded.

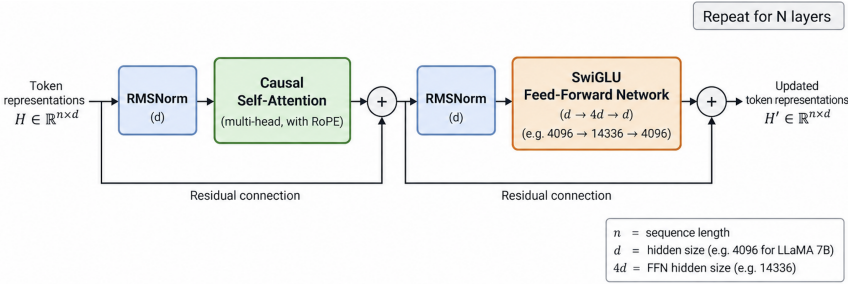
During **backpropagation**, the same blocks are recomputed from Q_i and K_j when they are needed again.

more arithmetic $\iff$ less memory and HBM traffic

MIXTURE-OF-EXPERTS I

FlashAttention makes attention more efficient. MoE (e.g. Mixtral ▸ Jiang et al., 2024: Mixtral) changes the **feed-forward part** of the Transformer.

$$H \to \text{Self-Attention} \to \tilde{H} \to \text{Feed-Forward Network (FFN)} \to H'.$$



- Self-attention remains dense.
- The single FFN is replaced by 8 **experts**. (Increase capacity.)
- For each token, only 2 experts are evaluated. (But use efficiently.)

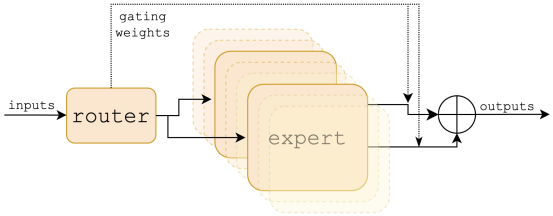
HOW DOES MIXTRAL SELECT EXPERTS? I

For each token representation $\tilde{h} \in \mathbb{R}^{4096}$ after self-attention, a small router computes 8 scores:

$$r = W_{\text{router}}\tilde{h}, \qquad W_{\text{router}} \in \mathbb{R}^{8\times 4096}.$$

- Select the top 2 experts.
- Both process the same token representation $\tilde{h}$.
- Their outputs are weighted and combined.

$$y = w_i E_i(\tilde{h}) + w_j E_j(\tilde{h}).$$



SPARSE ROUTING SAVES COMPUTE I

Each expert in Mixtral is a full SwiGLU FFN:

$$4096 \to 14336 \to 4096.$$

It has three projection matrices (including a gating projection):

▸ Shazeer, 2020

$$P_{\text{expert}} \approx 3 \cdot 4096 \cdot 14336 \approx 176\text{M}.$$

Mixtral stores 8 experts, but evaluates only 2 per token:

$$8 \times 176\text{M stored} \qquad \text{vs.} \qquad 2 \times 176\text{M evaluated}.$$

(The router adds only about $8 \cdot 4096 \approx 33{,}000$ parameters.)
Across the model:

47B total parameters	$\approx$ 13B active/token

More capacity without activating all parameters for every token.